
Problem Set 4

Grant Talbert

February 20, 2025

PY 451 Section A1

Boston University

1 Expectation Values etc.

1. This problem requires finding the eigenvalues and eigenvectors of M . This was done using a numpy script.

```
1 #!/bin/python3
2
3 import numpy as np
4
5 M = np.array([[5,0,0,0,0,0],
6               [0,0,3,0,0,0],
7               [0,3,0,0,0,0],
8               [0,0,0,0,2j,0],
9               [0,0,0,-2j,0,0],
10              [0,0,0,0,0,7]])
11
12 a, b = np.linalg.eigh(M) # chatgpt said eigh > eig for hermitian matrices
13
14 # get rid of floating point errors
15
16 # dont print the stupid +0j so its easier to like read
17 def format_complex(z):
18     if np.isclose(z.imag, 0):
19         return f"{z.real:.3f}"
20     else:
21         return f"{z.real:.3f} + {z.imag:.3f}j"
22
23 # print eigenvalues
24 for e in a:
25     print(format_complex(e))
26 print("-----")
27
28 # print eigenstates
29 f_b = np.array([[format_complex(z) for z in row] for row in b])
30 print(f_b)
```

It gives the following output:

```
1 -3.000
2 -2.000
3 2.000
4 3.000
5 5.000
6 7.000
7 -----
8 [['0.000' '0.000' '0.000' '0.000' '1.000' '0.000']
9  ['-0.707' '0.000' '0.000' '0.707' '0.000' '0.000']
10  ['0.707' '0.000' '0.000' '0.707' '0.000' '0.000']
11  ['0.000' '-0.707' '0.707' '0.000' '0.000' '0.000']
12  ['0.000' '0.000 + -0.707j' '0.000 + -0.707j' '0.000' '0.000' '0.000']
13  ['0.000' '0.000' '0.000' '0.000' '0.000' '1.000']]
```

The eigenvalues of M are

$$\{-3, -2, 2, 3, 5, 7\}.$$

The eigenvectors are

$$\left\{ \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}, \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}, \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}, \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ -1 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ -i \\ -i \\ 0 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} \right\}.$$

This set is clearly already normalized, and is also fairly obviously orthogonal (easily verified by computations that would be very lengthy and annoying to include). Therefore, these are all the possible results you can measure. I will denote these states $|m_i\rangle$, which will be ordered in the same order they appear above. I don't think you want to read 2 pages of me writing out inner products so I will only show the computational steps for the first one.

$$\mathcal{P}_{m_1} = |\langle m_1 | \psi \rangle|^2 = \left| \frac{1}{\sqrt{6}} \begin{pmatrix} 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{pmatrix} \right|^2 = \left| \frac{1}{\sqrt{6}} \right|^2 = \frac{1}{6};$$

$$\mathcal{P}_{m_2} = |\langle m_2 | \psi \rangle|^2 = 0; \quad \mathcal{P}_{m_3} = |\langle m_3 | \psi \rangle|^2 = \frac{1}{3};$$

$$\mathcal{P}_{m_4} = |\langle m_4 | \psi \rangle|^2 = 0; \quad \mathcal{P}_{m_5} = |\langle m_5 | \psi \rangle|^2 = \frac{1}{3};$$

$$\mathcal{P}_{m_6} = |\langle m_6 | \psi \rangle|^2 = \frac{1}{6}.$$

2. I will use the formula $(\Delta M)^2 = \langle M^2 \rangle - \langle M \rangle^2$, because I feel like it. First, we compute M^2 . Since M is a measurement operator, it is Hermitian. Moreover, since we are using an orthonormal set of eigenvectors, $\langle m_i | m_j \rangle = \delta_{ij}$, and thus

$$\begin{aligned} M &= \sum_i m_i |m_i\rangle \langle m_i| \\ \Rightarrow M^2 &= \left(\sum_i m_i |m_i\rangle \langle m_i| \right)^2 \\ &= \sum_i \left(m_i |m_i\rangle \langle m_i| \sum_j m_j |m_j\rangle \langle m_j| \right) \\ &= \sum_i m_i^2 |m_i\rangle \langle m_i| \\ &= \sum_i m_i^2 |m_i\rangle \langle m_i|. \end{aligned}$$

Here's some code to compute this.

```

1 #!/bin/python3
2
3 import numpy as np
4
5 M = np.array([[5,0,0,0,0,0],
6               [0,0,3,0,0,0],
7               [0,3,0,0,0,0],
8               [0,0,0,0,2j,0],
9               [0,0,0,-2j,0,0],
10              [0,0,0,0,0,7]])
11
12 a, b = np.linalg.eigh(M)
13
14 # compute the thingy
15 Msq = sum(a[i] * a[i] * np.outer(b[:, i], b[:, i].conj()) for i in range(len(a)))
16
17 # get rid of floating point errors
18 # dont print the stupid +0j or decimals so its easier to like read
19 def format_complex(z):
20     if np.isclose(z.imag, 0):
21         return f"{z.real:.0f}"
22     else:
23         return f"{z.real:.0f} + {z.imag:.0f}j"
24
25 Mfsq = np.array([[format_complex(z) for z in row] for row in Msq])
26 print(Mfsq)

```

Output:

```

1 [['25' '0' '0' '0' '0' '0']
2  ['0' '9' '0' '0' '0' '0']
3  ['0' '0' '9' '0' '0' '0']
4  ['0' '0' '0' '4' '0' '0']
5  ['0' '0' '0' '0' '4' '0']
6  ['0' '0' '0' '0' '0' '49']]

```

Therefore,

$$M^2 = \begin{pmatrix} 25 & & & & & \\ & 9 & & & & \\ & & 9 & & & \\ & & & 4 & & \\ & & & & 4 & \\ & & & & & 49 \end{pmatrix}.$$

In hindsight, since I wrote a script anyways, I should have just done $M @ M$ but oh well. Here's a script that computes the expectation of M and M^2 then subtracts them in the correct order.

```

1 #!/bin/python3
2
3 import numpy as np
4
5 M = np.array([[5,0,0,0,0,0],
6               [0,0,3,0,0,0],
7               [0,3,0,0,0,0],
8               [0,0,0,0,2j,0],
9               [0,0,0,-2j,0,0],
10              [0,0,0,0,0,7]])

```

```

11
12 Msq = M @ M
13
14 psi = (1/np.sqrt(6)) * np.ones((6, 1), dtype=complex)
15
16 EV_M = psi.conj().T @ M @ psi
17 EV_Msq = psi.conj().T @ Msq @ psi
18
19 stdev = np.sqrt(EV_Msq - EV_M)
20
21 print(stdev)

```

Output:

```

1 [[3.6968455+0.j]]

```

Therefore, $\Delta M \approx 3.6968455$. Now to do the same for A . Here is a block of code that just calculates the standard deviation.

```

1 #!/bin/python3
2
3 import numpy as np
4
5 A = np.array([[0,1,0,0,0,0],
6               [1,0,0,0,0,0],
7               [0,0,0,4j,0,0],
8               [0,0,-4j,0,0,0],
9               [0,0,0,0,3,0],
10              [0,0,0,0,0,2]])
11
12 Asq = A @ A
13
14 psi = (1/np.sqrt(6)) * np.ones((6, 1), dtype=complex)
15
16 EV_A = psi.conj().T @ A @ psi
17 EV_Asq = psi.conj().T @ Asq @ psi
18
19 stdev = np.sqrt(EV_Asq - EV_A)
20
21 print(stdev)

```

Output:

```

1 [[2.5819889+0.j]]

```

Therefore, $\Delta A \approx 2.5819889$. Here is the calculation of $|\langle [A, M] \rangle|$:

```

1 #!/bin/python3
2
3 import numpy as np
4
5 A = np.array([[0,1,0,0,0,0],
6               [1,0,0,0,0,0],
7               [0,0,0,4j,0,0],
8               [0,0,-4j,0,0,0],
9               [0,0,0,0,3,0],
10              [0,0,0,0,0,2]])
11
12 M = np.array([[5,0,0,0,0,0],
13               [0,0,3,0,0,0],

```

```

14         [0,3,0,0,0,0],
15         [0,0,0,0,2j,0],
16         [0,0,0,-2j,0,0],
17         [0,0,0,0,0,7]])
18
19 psi = (1/np.sqrt(6)) * np.ones((6, 1), dtype=complex)
20 comm = (A @ M) - (M @ A)
21 EV = psi.conj().T @ comm @ psi
22
23 print(np.abs(EV))

```

Output:

```

1      [[6.]]

```

The uncertainty principle states

$$(\Delta A)(\Delta M) \geq \frac{1}{2} |\langle [A, M] \rangle|.$$

Note that 3 is clearly less than the product $3.7 \cdot 2.6$.

3. (c) is correct, because if $\Delta A, \Delta M = 0$, then $(\Delta A)(\Delta M) = 0$, and this implies that $\langle A, M \rangle = 0$. However, the matrices do not commute:

```

1  #!/bin/python3
2
3  import numpy as np
4
5  A = np.array([[0,1,0,0,0,0],
6               [1,0,0,0,0,0],
7               [0,0,0,4j,0,0],
8               [0,0,-4j,0,0,0],
9               [0,0,0,0,3,0],
10              [0,0,0,0,0,2]])
11
12 M = np.array([[5,0,0,0,0,0],
13              [0,0,3,0,0,0],
14              [0,3,0,0,0,0],
15              [0,0,0,0,2j,0],
16              [0,0,0,-2j,0,0],
17              [0,0,0,0,0,7]])
18
19 print(A @ M - M @ A)

```

Output:

```

1      [[ 0. +0.j -5. +0.j  3. +0.j  0. +0.j  0. +0.j  0. +0.j]
2      [ 5. +0.j  0. +0.j  0. +0.j  0. -12.j  0. +0.j  0. +0.j]
3      [-3. +0.j  0. +0.j  0. +0.j  0. +0.j -8. +0.j  0. +0.j]
4      [ 0. +0.j  0. -12.j  0. +0.j  0. +0.j  0. -6.j  0. +0.j]
5      [ 0. +0.j  0. +0.j  8. +0.j  0. -6.j  0. +0.j  0. +0.j]
6      [ 0. +0.j  0. +0.j  0. +0.j  0. +0.j  0. +0.j  0. +0.j]]

```

2 Complete Eigenbasis

1. This code checks whether they are eigenvectors, and prints the eigenvalue if they are.

```

1 #!/bin/python3
2
3 import numpy as np
4
5 H = np.array([[31/12, 3/4, 13/12, 7/12],
6               [3/4, 11/4, 3/4, 3/4],
7               [13/12, 3/4, 25/12, 13/12],
8               [7/12, 3/4, 13/12, 31/12]])
9 v1 = np.ones((4, 1))
10 v2 = np.array([[ -5/2], [3/2], [ -1/2], [3/2]])
11 v3 = np.array([[ -np.sqrt(3)/4], [np.sqrt(3)/4], [ -1/(4 * np.sqrt(3))], [1/(4 *
12               np.sqrt(3))]])
13 v4 = np.array([[ -1], [0], [2], [ -1]])
14
15 # check if all list elements r same
16 def all_same(vals):
17     first = vals[0]
18     return all(np.allclose(x, first) for x in vals) # thanks chatgpt
19
20 # check if eigenvalue
21 def is_ev(v):
22     mat = H @ v
23     vals = []
24     for i in range(1, len(v)):
25         # handle div by 0
26         if v[i, 0] == 0:
27             if mat[i, 0] != 0:
28                 print("failed")
29                 return False
30             else:
31                 vals.append(mat[i, 0] / v[i, 0])
32     if all_same(vals) == 1:
33         return vals[0]
34     else:
35         print("failed")
36         return False
37
38 # print evals if it works and print something else if it doesnt
39 if is_ev(v1):
40     print(f"v1: {is_ev(v1)}")
41 if is_ev(v2):
42     print(f"v2: {is_ev(v2)}")
43 if is_ev(v3):
44     print(f"v3: {is_ev(v3)}")
45 if is_ev(v4):
46     print(f"v4: {is_ev(v4)}")

```

Output:

```

1 v1: 5.0
2 v2: 2.0
3 v3: 2.0
4 v4: 1.0000000000000004

```

We have that the eigenvalues are

$$v_1 = 5, \quad v_2 = 2, \quad v_3 = 2, \quad v_4 = 1.$$

The matrix representation in the eigenbasis is then just

$$\begin{pmatrix} 5 & & & \\ & 2 & & \\ & & 2 & \\ & & & 1 \end{pmatrix}.$$

2. The vectors span the space if and only if

$$c_1|v_1\rangle + c_2|v_2\rangle + c_3|v_3\rangle + c_4|v_4\rangle$$

implies $c_i = 0$ for all i . Equivalently, the equation

$$\begin{pmatrix} |v_1\rangle & |v_2\rangle & |v_3\rangle & |v_4\rangle \end{pmatrix} |c\rangle = 0$$

is solved if and only if $|c\rangle = 0$. For simplicity, call the matrix of kets V . This happens if and only if $\ker V$ is trivial; i.e. the linear map given by the matrix is injective, and thus is bijective since our space is 4-dimensional. It follows that V must have rank 4, which can be computed by numpy.

```
1 #!/bin/python3
2
3 import numpy as np
4
5 v1 = np.array([1,1,1,1], dtype=complex)
6 v2 = np.array([-5/2, 3/2, -1/2, 3/2], dtype=complex)
7 v3 = np.array([-np.sqrt(3)/4, np.sqrt(3)/4, -1/(4 * np.sqrt(3)), 1/(4 * np.
8     sqrt(3))], dtype=complex)
9 v4 = np.array([-1, 0, 2, -1], dtype=complex)
10 print(np.linalg.matrix_rank(np.column_stack([v1,v2,v3,v4])))
```

Output:

```
1 4
```

Therefore, the eigenstates span the space.

3. I'm going to use Gram-Schmidt. Start with

$$|u_1\rangle = \frac{1}{2} \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}.$$

Since $|v_1\rangle$ and $|v_2\rangle$ are orthogonal,

$$|u_2\rangle = \frac{1}{2\sqrt{11}} \begin{pmatrix} -5 \\ 3 \\ -1 \\ 3 \end{pmatrix}.$$

I'm crashing out I'm done please give me a 0 on everything else